1η Ομαδική Εργασία
Σχεδίαση Ψηφιακών Συστημάτων

Πρόβλημα 1

1. Προσδιορισμός των μορφών SOP και POS ελαχίστου κόστους της συνάρτησης με χρήση πινάκων Karnaugh και σύγκριση των μεταξύ τους κοστών.

Για την συνάρτηση πέντε μεταβλητών:
f(x1,x2,x3,x4,x5) = Σm(1,2,5,8,9,12,14,17,19,21,27) + D(0,6,11,23,24,29,30)
έχουμε τους παρακάτω πίνακες Karnaugh, που δείχνουν τα minterms:

x3 x4 \ x1 x2	00	01	11	10
00	m0	m8	m24	m16
01	m2	m10	m26	m18
11	m6	m14	m30	m22
10	m4	m12	m28	m20

x5=0

x3 x4 \ x1 x2	00	01	11	10
00	m1	m9	m25	m17
01	m3	m11	m27	m19
11	m7	m15	m31	m23
10	m5	m13	m29	m21

x5=1

Στην συνέχεια τοποθετούμε 1 στα κελιά που συμμετέχουν στο άθροισμα minterms και D στα κελιά don’t care:

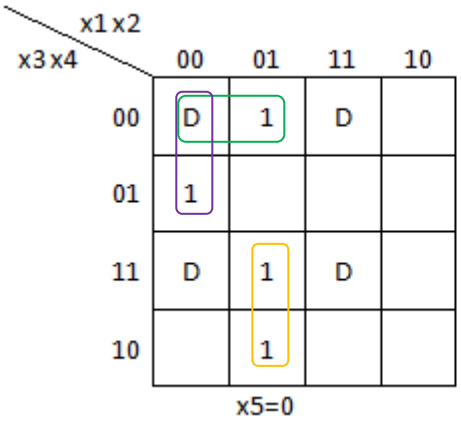
x3 x4 \ x1 x2	00	01	11	10
00	D	1	D	
01	1			
11	D	1	D	
10		1		

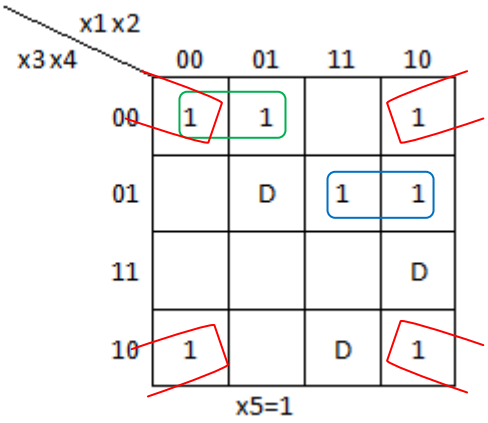
x5=0

x3 x4 \ x1 x2	00	01	11	10
00	1	1		1
01		D	1	1
11				D
10	1		D	1

x5=1

Για την μορφή **SOP**, συνδυάζουμε τα κελιά όπως παρακάτω:





Και έχουμε:

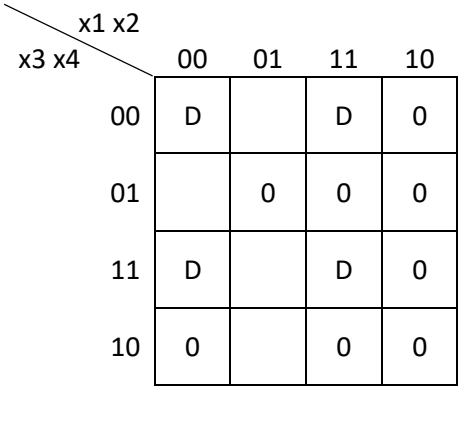
Ομάδα κελιών m1,m5,m17,m21, όρος του αθροίσματος: x2' x4' x5
Ομάδα κελιών m0,m1,m8,m9, όρος του αθροίσματος: x1' x3' x4'
Ομάδα κελιών m12,m14, όρος του αθροίσματος: x1' x2 x3 x5'
Ομάδα κελιών m19,m27, όρος του αθροίσματος: x1 x3' x4 x5
Ομάδα κελιών m0,m2, όρος του αθροίσματος: x1' x2' x3' x5'

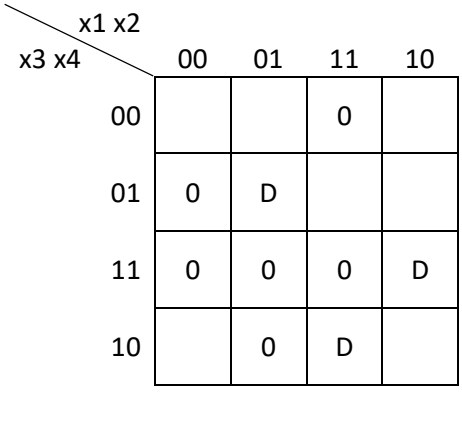
Άρα η f απλοποιείται στην:

f= x2' x4' x5 + x1' x3' x4' + x1' x2 x3 x5' + x1 x3' x4 x5 + x1' x2' x3' x5'

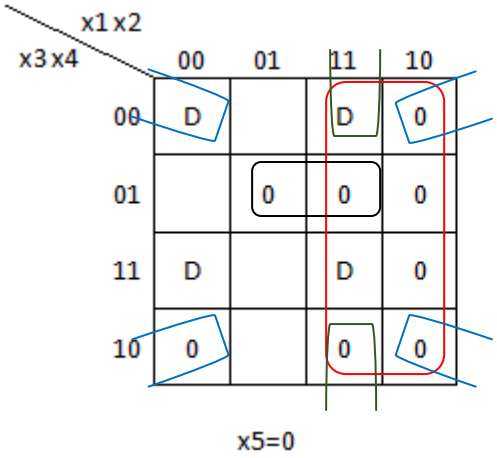
Για το κόστος συνυπολογίζουμε το πλήθος των εισόδων των πυλών και το πλήθος των πυλών, έτσι έχουμε:
2 AND των 3 εισόδων = 6
3 AND των 4 εισόδων= 12
1 OR των 5 εισόδων= 5
Σύνολο πυλών= 6
Άρα **κόστος SOP = 6+12+5+6=29**

Για την μορφή **POS**, παίρνουμε τα κελιά με 0 και αυτά με τα don't care, για να βρούμε την f':

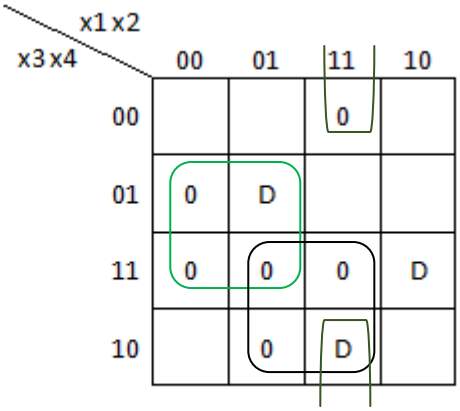




και τα συνδυάζουμε όπως παρακάτω:



x5=0



x5=1

Και έχουμε:

Ομάδα κελιών	m16,m18,m20,m22,m24,m26,m28,m30, όρος του αθροίσματος: x1 x5'
Ομάδα κελιών	m0,m4,m16,m20, όρος του αθροίσματος: x2' x4' x5'
Ομάδα κελιών	m3,m7,m11, m15, όρος του αθροίσματος: x1' x4 x5
Ομάδα κελιών	m13,m15,m29,m31, όρος του αθροίσματος: x2 x3 x5
Ομάδα κελιών	m24,m25,m28,m29 όρος του αθροίσματος: x1 x2 x4'
Ομάδα κελιών	m10,m26, όρος του αθροίσματος: x2 x3' x4 x5'

Άρα η f' απλοποιείται στην:

f'= x1 x5' + x2' x4' x5' + x1' x4 x5 + x2 x3 x5 + x1 x2 x4' + x2 x3' x4 x5', έτσι

(f')'=(x1 x5' + x2' x4' x5' + x1' x4 x5 + x2 x3 x5 + x1 x2 x4' + x2 x3' x4 x5')' και από de Morgan

f=(x1'+x5) (x2+x4+x5) (x1+x4'+x5') (x2'+x3'+x5') (x1'+x2'+x4) (x2'+x3+x4'+x5)

Για το κόστος έχουμε:

1 OR των2 εισόδων =	2
4 OR των 3 εισόδων=	12
1 OR των 4 εισόδων=	4
1 AND των 6 εισόδων=	6
Σύνολο πυλών=	7

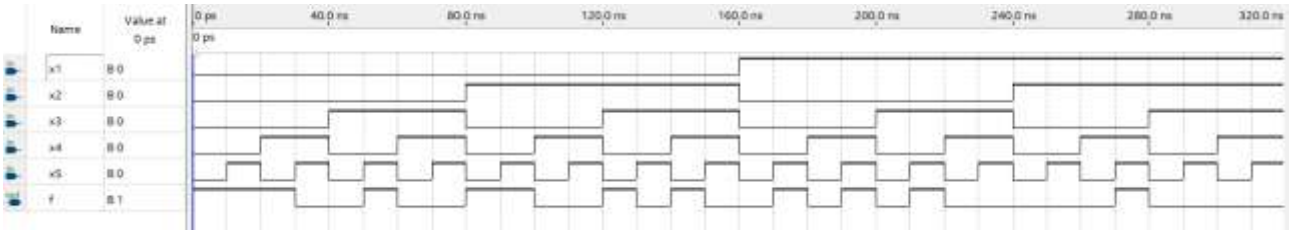
Άρα **κόστος POS = 2+12+4+6+7=31**

Άρα το μικρότερο κόστος το έχει η υλοποίηση SOP και αυτή θα υλοποιήσουμε στο επόμενο ερώτημα με το λογισμικό QUARTUS

2. Χρησιμοποιούμε το λογισμικό Quartus, για να γράψουμε πηγαίο κώδικα (πρόγραμμα) στη γλώσσα VHDL (File → New → Design Files → VHDL File) , ο οποίος να υλοποιεί το κύκλωμα με την μορφή SOP που βρήκαμε στο ερώτημα 1.
   f= x2' x4' x5 + x1' x3' x4' + x1' x2 x3 x5' + x1 x3' x4 x5 + x1' x2' x3' x5'
   Ο φάκελος του project είναι ο **HW1_2**
   a. Ο κώδικας ακολουθεί structural προσέγγιση και υλοποιούνται ως υποκυκλώματα (components) οι όροι της συνάρτησης f. Το αρχείο είναι το **HW1_2.qpf.** Στη συνέχεια κάνουμε Compilation (Processing → Start Compilation)

   b. Χρησιμοποιούμε την προσομοίωση λειτουργίας του λογισμικού Quartus για να αποδείξουμε την ορθότητα της υλοποίησής μας (File → New → Verification/Debugging Files → University Program VWF). Παράγεται το αρχείο **Waveform.vwf** , μέσα στον φάκελο του project. Η κυματομορφή της

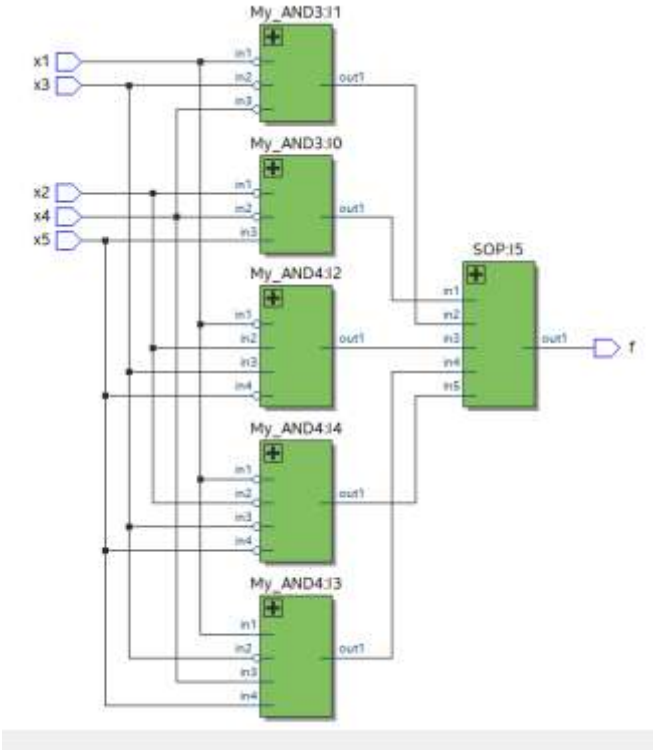
λειτουργικής προσομοίωσης εξόδου για όλες τις δυνατές τιμές εισόδου (ανά 10nsec) είναι η παρακάτω (υπάρχει ως screenshot στο αρχείο **1_2_b waveform.jpg**):



Η ορθότητα της υλοποίησης προκύπτει συγκρίνοντας τις τιμές της παραπάνω κυματομορφής με τον πίνακα αλήθειας της αρχικής συνάρτησης που φαίνεται παρακάτω:

	x1	x2	x3	x4	x5	f
0	0	0	0	0	0	x
1	0	0	0	0	1	1
2	0	0	0	1	0	1
3	0	0	0	1	1	0
4	0	0	1	0	0	0
5	0	0	1	0	1	1
6	0	0	1	1	0	x
7	0	0	1	1	1	0
8	0	1	0	0	0	1
9	0	1	0	0	1	1
10	0	1	0	1	0	0
11	0	1	0	1	1	x
12	0	1	1	0	0	1
13	0	1	1	0	1	0
14	0	1	1	1	0	1
15	0	1	1	1	1	0
16	1	0	0	0	0	0
17	1	0	0	0	1	1
18	1	0	0	1	0	0
19	1	0	0	1	1	1
20	1	0	1	0	0	0
21	1	0	1	0	1	1
22	1	0	1	1	0	0
23	1	0	1	1	1	x
24	1	1	0	0	0	x
25	1	1	0	0	1	0
26	1	1	0	1	0	0
27	1	1	0	1	1	1
28	1	1	1	0	0	0
29	1	1	1	0	1	x
30	1	1	1	1	0	x
31	1	1	1	1	1	0

c. Το RTL διάγραμμα (Tools→Netlist Viewers→RTL viewer) που προκύπτει (υπάρχει ως screenshot στο αρχείο **1_2_c RTL.pdf**):



3. Υπολογισμός μορφής SOP ελαχίστου κόστους της συνάρτησης , με χρήση της μεθόδου **Quine-McCluskey**.

Δημιουργία πρώτων όρων αλήθειας για την συνάρτηση
f(x1,x2,x3,x4,x5) = Σm(1,2,5,8,9,12,14,17,19,21,27) + D(0,6,11,23,24,29,30)

Πίνακας 1

Αρ. 1	minterm	Δυαδική Αναπαράσταση	
0	0	00000	✓
1	1	00001	✓
	2	00010	✓
	8	01000	✓
2	5	00101	✓
	6	00110	✓
	9	01001	✓
	12	01100	✓
	17	10001	✓
	24	11000	✓
3	11	01011	✓
	14	01110	✓
	19	10011	✓
	21	10101	✓
4	23	10111	✓
	27	11011	✓
	29	11101	✓
	30	11110	✓

Πίνακας 2

Αρ. 1	Συνδυασμός 2 minterm	Δυαδική Αναπαράσταση	
0	0,1	0000x	✓
	0,2	000x0	
	0,8	0x000	✓
1	1,5	00x01	✓
	1,9	0x001	✓
	1,17	x0001	✓
	2,6	00x10	
	8,9	0100x	✓
	8,12	01x00	
	8,24	x1000	
2	5,21	x0101	✓
	6,14	0x110	
	9,11	010x1	
	12,14	011x0	
	17,19	100x1	✓
	17,21	10x01	✓
	11,27	x1011	
3	14,30	x1110	
	19,23	10x11	✓
	19,27	1x011	
	21,23	101x1	✓
	21,29	1x101	

Πίνακας 3

Αρ. 1	Συνδυασμός 4 minterm	Δυαδική Αναπαράσταση
0	0,1,8,9	0x00x
1	1,5,17,21	x0x01
2	17,19,21,23	10xx1

Επιλογή κάλυψης:

Πρώτος όρος αλήθειας	Minterm										
	1	2	5	8	9	12	14	17	19	21	27
p1=0x00x	✓			✓	✓						
p2= x0x01	✓		✓					✓		✓	
p3=10xx1								✓	✓	✓	
p4=000x0		✓									
p5=00x10		✓									
p6=01x00				✓		✓					
p7= x1000				✓							
p8=0x110							✓				
p9=010x					✓						
p10=011x0						✓	✓				
p12= x1011											✓
p13= x1110							✓				
p14=1x011									✓		✓
p15=1x101										✓	

Παρατηρούμε ότι στον παραπάνω πίνακα η στήλη του minterm 5 καλύπτεται μόνο από τον όρο **p2** που είναι ουσιαστικός πρώτος όρος και πρέπει να συμπεριληφθεί στην τελική κάλυψη.

Ο όρος p5 μπορεί να απομακρυνθεί διότι καλύπτεται από τον **p4**, ο οποίος καθίσταται ουσιαστικός και πρέπει να συμπεριληφθεί στην τελική κάλυψη

Πρώτος όρος αλήθειας	Minterm										
	1	2	5	8	9	12	14	17	19	21	27
p1=0x00x	✓			✓	✓						
p2= x0x01	✓		✓					✓		✓	
p3=10xx1								✓	✓	✓	
p4=000x0		✓									
p6=01x00				✓		✓					
p7= x1000				✓							
p8=0x110							✓				
p9=010x					✓						
p10=011x0						✓	✓				
p12= x1011											✓
p13= x1110							✓				
p14=1x011									✓		✓
p15=1x101										✓	

Ο όρος p12 μπορεί να απομακρυνθεί γιατί καλύπτει μόνο τον minterm 27 που καλύπτεται από τον όρο p14, ο οποίος καλύπτει και τον minterm 19 και έτσι ο όρος p14 καθίσταται ουσιαστικός και πρέπει να συμπεριληφθεί στην τελική κάλυψη.

Πρώτος όρος αλήθειας	Minterm										
	1	2	5	8	9	12	14	17	19	21	27
p1=0x00x	✓			✓	✓						
p2= x0x01	✓		✓					✓		✓	
p3=10xx1								✓	✓	✓	
p4=000x0		✓									
p6=01x00				✓		✓					
p7= x1000				✓							
p8=0x110							✓				
p9=010x					✓						
p10=011x0						✓	✓				
p13= x1110							✓				
p14=1x011									✓		✓
p15=1x101										✓	

Αφού οι p2 και p14 είναι ουσιαστικοί, ο p3 μπορεί να απομακρυνθεί γιατί καλύπτεται από αυτούς, ομοίως και ο p15.

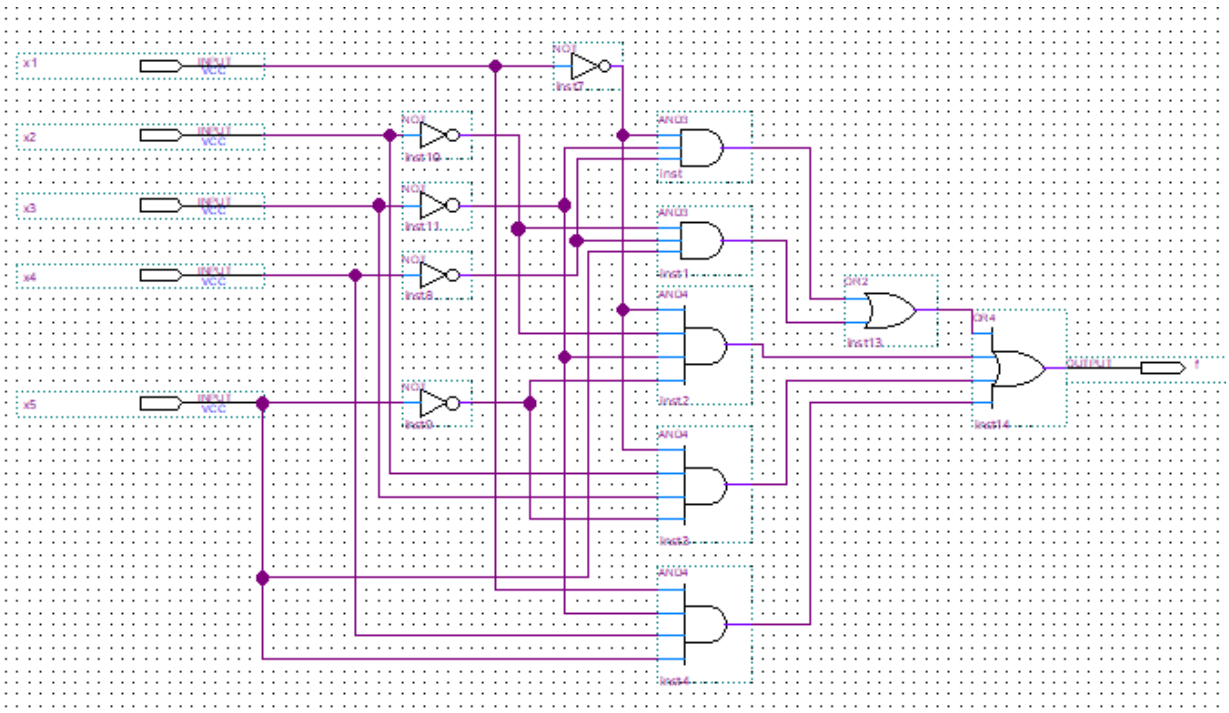
Πρώτος όρος αλήθειας	Minterm										
	1	2	5	8	9	12	14	17	19	21	27
p1=0x00x	✓			✓	✓						
p2= x0x01	✓		✓					✓		✓	
p4=000x0		✓									
p6=01x00				✓		✓					
p7= x1000				✓							
p8=0x110							✓				
p9=010x					✓						
p10=011x0						✓	✓				
p13= x1110							✓				
p14=1x011									✓		✓

Τώρα μένουν να καλυφθούν οι minterms 8, 9, 12 και 14. Αυτό μπορεί να γίνει με τους p1 και p10. Έτσι οι p6, p7, p8, p9 και p13, μπορούν να απομακρυνθούν.

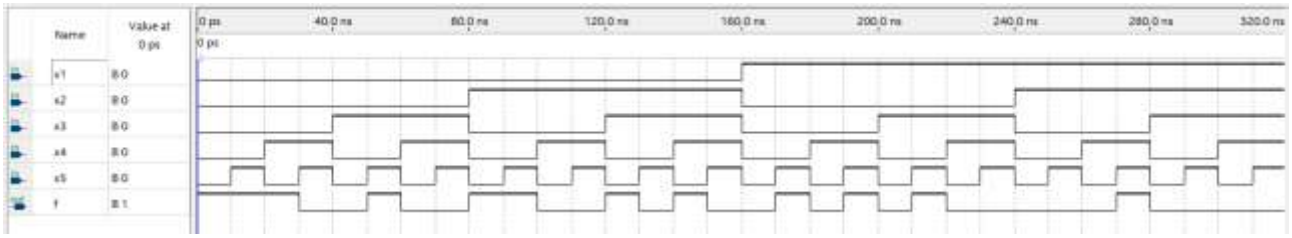
Πρώτος όρος αλήθειας	Minterm										
	1	2	5	8	9	12	14	17	19	21	27
p1=0x00x	✓			✓	✓						
p2= x0x01	✓		✓					✓		✓	
p4=000x0		✓									
p10=011x0						✓	✓				
p14=1x011									✓		✓

Τελικά f= p1 + p2 + p4 + p10 + p14 δηλαδή
f= x1'x3'x4' + x2'x4'x5 + x1'x2'x3'x5' +x1'x2x3x5' + x1x3'x4x5
Που είναι **ίδια** με αυτή που βρήκαμε στο ερώτημα 1.

a. Χρησιμοποιούμε το λογισμικό Quartus, για να σχεδιάσουμε το **σχηματικό διάγραμμα** (block/schematic diagram), το οποίο υλοποιεί το κύκλωμα που βρήκαμε στο ερώτημα 3. Ο φάκελος του project είναι ο **HW1_3.** Το αρχείο που δημιουργείται είναι το **HW1_3.bdf**.



b. Χρησιμοποιούμε την προσομοίωση λειτουργίας του λογισμικού Quartus για να αποδείξουμε την ορθότητα της υλοποίησής μας (File → New → Verification/Debugging Files → University Program VWF). Παράγεται το αρχείο Waveform.vwf , μέσα στον φάκελο του project. Η κυματομορφή της λειτουργικής προσομοίωσης εξόδου για όλες τις δυνατές τιμές εισόδου (ανά 10nsec) είναι η παρακάτω (υπάρχει ως screenshot στο αρχείο **1_3_b waveform.jpg**):



Παρατηρούμε ότι η κυματομορφή είναι ίδια με αυτήν του ερωτήματος 2.b.

Πρόβλημα 2

1. Για να εξάγουμε την συνάρτηση (που περιγράφει το κύκλωμα) που το δημιουργεί με την απλούστερη μορφή γινομένου αθροισμάτων (POS) με χρήση πινάκων Karnaugh, ακολουθούμε τα παρακάτω βήματα:

Αρχικά, χρησιμοποιούμε το διάγραμμα χρονισμού για να φτιάξουμε τον πίνακα αληθείας της συνάρτησης.

Για να το κάνουμε αυτό, παρατηρούμε στο διάγραμμα χρονισμού τα αποτελέσματα του κάθε κύκλου. Έπειτα, βρίσκουμε σε ποιο συνδυασμό x1,x2,x3,x4 ανήκει το αποτέλεσμα και το γράφουμε στην f.

x1	x2	x3	x4	f
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	0
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	0
1	1	1	0	1
1	1	1	1	1

Συνεχίζουμε βρίσκοντας τα mnterms και τα maxterms:

Min Terms	Max Terms
x1' * x2' * x3' * x4'	x1 + x2 + x3 + x4
x1' * x2' * x3' * x4	x1 + x2 + x3 + x4'
x1' * x2' * x3 * x4'	x1 + x2 + x3' + x4
x1' * x2' * x3 * x4	x1 + x2 + x3' + x4'
x1' * x2 * x3' * x4'	x1 + x2' + x3 + x4
x1' * x2 * x3' * x4	x1 + x2' + x3 + x4'
x1' * x2 * x3 * x4	x1 + x2' + x3' + x4
x1' * x2 * x3 * x4	x1 + x2' + x3' + x4'
x1 * x2' * x3' * x4'	x1' + x2 + x3 + x4
x1 * x2' * x3' * x4	x1' + x2 + x3 + x4'
x1 * x2' * x3 * x4'	x1' + x2 + x3' + x4
x1 * x2' * x3 * x4	x1' + x2 + x3' + x4'
x1 * x2 * x3' * x4'	x1' + x2' + x3 + x4
x1 * x2 * x3' * x4	x1' + x2' + x3 + x4'
x1 * x2 * x3 * x4'	x1' + x2' + x3' + x4
x1 * x2 * x3 * x4	x1' + x2' + x3' + x4'

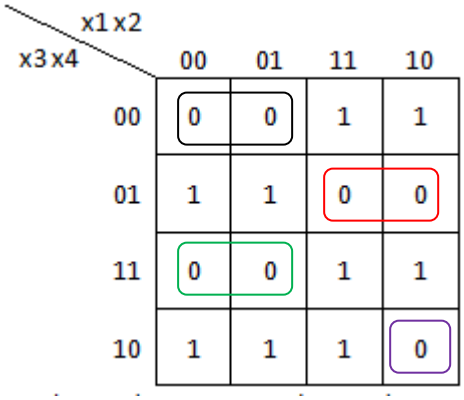
Καθώς θέλουμε μορφή γινομένου αθροισμάτων (POS), θέλουμε τα Max Terms. Συνεπώς, πάμε στον πίνακα αληθείας και παίρνουμε μόνο τα x1,x2,x3,x4 που έχουν ως αποτέλεσμα 0.
Έτσι, η αρχική μας συνάρτηση είναι η εξής:

f=(x1 + x2 + x3 +x4) (x1 + x2 + x3' + x4') (x1 + x2' + x3 + x4) (x1 + x2' + x3' + x4') (x1' + x2 + x3 + x4') (x1' + x2 + x3' + x4) (x1' + x2' + x3 + x4')

Για να απλοποιήσουμε την αρχική μας συνάρτηση, θα χρησιμοποιήσουμε πίνακα Karnaugh:

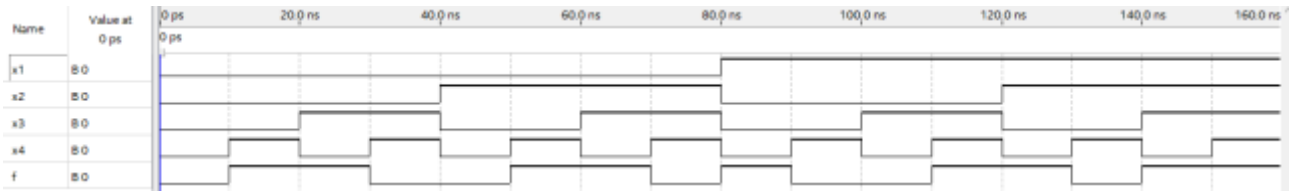
x3 x4 \ x1 x2	00	01	11	10
00	0	0	1	1
01	1	1	0	0
11	0	0	1	1
10	1	1	1	0

Και εφόσον θέλουμε μορφή γινομένου αθροισμάτων, θα πρέπει να συνδυάσουμε σε όρους αθροισμάτων τους όρους maxterm για τους οποίους το f είναι 0. Έτσι συνδυάζουμε τα κελιά όπως παρακάτω:



Και παίρνουμε: **f= (x1 + x3 + x4) (x1' + x3 + x4') (x1 + x3' + x4') (x1' + x2 + x3' + x4)**

2. Χρησιμοποιούμε το λογισμικό Quartus, για να γράψουμε πηγαίο κώδικα (πρόγραμμα) στη γλώσσα VHDL (File → New → Design Files → VHDL File) , ο οποίος να υλοποιεί το κύκλωμα (συνάρτηση) που βρήκαμε στο ερώτημα 1.
   Ο φάκελος του project είναι ο **tut_vhdl**. Το αρχείο κώδικα είναι το **tut_vhdl.vhd**, στη συνέχεια κάνουμε compilation.
   a. Χρησιμοποιούμε την προσομοίωση λειτουργίας του λογισμικού Quartus για να επιβεβαιώσουμε ότι η κυματομορφή λειτουργίας που προκύπτει με βάση τον κώδικα VHDL είναι ίδια με το δοσμένο διάγραμμα, (File → New → Verification/Debugging Files → University Program VWF). Παράγεται το αρχείο **Waveform.vwf** , μέσα στον φάκελο του project. Η κυματομορφή της λειτουργικής προσομοίωσης εξόδου για όλες τις δυνατές τιμές εισόδου (ανά 10nsec) είναι η παρακάτω (υπάρχει ως screenshot στο αρχείο **2_2_a waveform.jpg**):



Παρατηρούμε ότι η παραπάνω κυματομορφή είναι ίδια με το δοσμένο διάγραμμα

c. Το RTL διάγραμμα (Tools/Netlist Viewers/RTL viewer) που προκύπτει (υπάρχει ως screenshot στο αρχείο **2_2_b RTL.pdf**):

